TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN – ĐHQG-HCM
KHOA KỸ THUẬT MÁY TÍNH

IT012 – TỔ CHỨC VÀ CẤU TRÚC MÁY TÍNH II

LAB 02

ỨNG DỤNG CỦA MẠCH SỐ

Họ và tên	Mai Gia Bảo	**MSSV**	25520138
Lớp	IT012.R11.1	**Ngày thực hành**	05/10/2026
Tự chấm	10	**Link git**	https://github.com/Giabao1937/IT012.R11.1_MaiGiaBao_25520138.git

1. Tổng quan buổi thực hành

Sinh viên dùng phần mềm **Logisim** để biến kiến thức Chương 4 (mạch số, mạch tổ hợp, mạch tuần tự, latch, flip-flop, thanh ghi) và Chương 5 (bộ cộng, ALU, bộ chọn, bộ so sánh, bộ giải mã, tập thanh ghi) thành một bài toán xuyên suốt: **tự xây một datapath 8 bit của CPU thu nhỏ**, từng khối một, theo chu trình **dự đoán – dựng mạch – quan sát – giải thích**.

Sau buổi thực hành, Sinh viên có thể:

- Thiết kế và dựng bộ cộng/trừ từ cổng luận lý theo quy trình 4 bước (bảng chân trị → biểu thức → tối ưu → sơ đồ), đọc được các **cờ** Z, N, C, V và phát hiện **tràn số**.
- Dựng **ALU 8 bit** từ bộ chọn (MUX), bộ cộng/trừ và cổng luận lý; dùng ALU để so sánh hai số và chỉ ra **giới hạn** của quy tắc so sánh bằng phép trừ.
- Phân biệt **latch** (tích cực theo mức) với **flip-flop** (tích cực theo cạnh) bằng dạng sóng; tự dựng D latch và ghép hai latch thành D flip-flop.
- Giải thích vì sao thanh ghi dùng flip-flop và vì sao vòng hồi tiếp "thanh ghi → ALU → thanh ghi" vẫn hoạt động đúng.
- Dựng **tập thanh ghi 4 × 8 bit**, ghép với ALU thành đường dữ liệu và điều khiển bằng tay để chạy một chương trình ngắn (C = A + B, hoán đổi hai thanh ghi).

Phần	Nội dung	Thời gian gợi ý	Điểm
Phần 0	Làm quen bus nhiều bit, mạch con, clock; kiểm tra môi trường	15 phút	
TN 1	Bộ cộng/trừ 4 bit	45 phút	2,0
TN 2	ALU 8 bit và bộ so sánh	45 phút	1,5
TN 3	Mạch tuần tự: D latch, D flip-flop	45 phút	3,0
TN 4	Tập thanh ghi và đường dữ liệu 8 bit	60 phút	3,0
Kết	Tổng kết, phản tư, nộp bài	15 phút	0,5
Tổng		**225 phút**	**10,0**

Quy ước làm bài

- **Ô nền vàng nhạt** là chỗ Sinh viên điền hoặc trả lời trực tiếp vào file này (gõ chữ, hoặc dán ảnh chụp màn hình vào khung nét đứt).
- Ký hiệu: NOT là dấu phẩy trên (A'); AND viết liền hoặc A·B; OR là A + B; XOR là A ⊕B. Số thập lục phân viết 0x3C. Với bus nhiều bit, A[7] là bit trọng số cao nhất (MSB). Số có dấu dùng biểu diễn **bù 2** (Chương 2).
- Mỗi thí nghiệm theo chu trình **DỰ ĐOÁN → QUAN SÁT → GIẢI THÍCH**. Ghi dự đoán **trước** khi mô phỏng; dự đoán sai **không bị trừ điểm**, phần giải thích vì sao sai mới được chấm.
- Các mạch con (subcircuit) tạo ở thí nghiệm trước được **dùng lại** ở thí nghiệm sau. Đặt tên đúng quy ước và không xoá.

PHẦN 0 — Làm quen bus, mạch con và clock (15 phút)

Sinh viên đã dùng Logisim ở Lab 01; phần này chỉ bổ sung các kỹ năng mới cho mạch nhiều bit và mạch tuần tự. Giảng viên hướng dẫn nhanh theo tài liệu *Hướng dẫn sử dụng Logisim*. Sinh viên tự đánh dấu ☑ khi thực hiện được từng kỹ năng (bỏ qua nếu đã thành thạo).

☑	Kỹ năng	Gợi ý
☐	Tạo project mới; đặt tên circuit đầu tiên là **Warmup**; lưu file **IT012_Lab02_<MSSV>.circ**	Mỗi thí nghiệm dùng các circuit riêng trong **cùng một file** (Project ▸ Add Circuit); tên circuit theo quy ước ở đầu mỗi thí nghiệm.
☐	Đặt Input Pin / Output Pin **nhiều bit** (thuộc tính Data Bits); đổi từng bit bằng Poke tool	Pin 8 bit hiển thị 8 ô bit. Dùng **Probe** (thư viện Wiring) và đổi thuộc tính **Radix** để đọc ở dạng nhị phân, thập lục phân, thập phân có dấu.
☐	Dùng **Splitter** để tách bus thành từng dây và ghép các dây thành bus	Fan Out = số dây, Bit Width In = độ rộng bus. Kiểm tra thứ tự bit (bit 0 là LSB).
☐	Tạo **mạch con**: dựng một circuit, rồi kéo circuit đó từ ngăn Explorer vào circuit khác như một linh kiện	Vị trí các pin trong circuit quyết định vị trí các chân của linh kiện; đặt nhãn pin rõ ràng.
☐	Dùng linh kiện có sẵn: **Plexers** (Multiplexer, Decoder), **Arithmetic** (Adder, Subtractor), **Memory** (D Flip-Flop, Register)	Chọn linh kiện rồi chỉnh Data Bits, Select Bits ở ngăn thuộc tính (góc dưới bên trái).
☐	Điều khiển mạch tuần tự: đổi tay một Input Pin **CLK**; dùng **Simulate ▸ Reset Simulation** để đưa mạch về trạng thái ban đầu	Cạnh lên xảy ra khi Poke CLK từ 0 lên 1. Với linh kiện Clock (Wiring), mỗi lần Simulate ▸ Tick Once đổi Clock một nửa chu kỳ.
☐	(Tuỳ chọn) Xem dạng sóng: **Chronogram** (Logisim-evolution) hoặc cửa sổ **Simulate ▸ Logging** (Logisim 2.7.x)	Tuỳ phiên bản; không bắt buộc, dùng để kiểm tra lại Bảng 10 ở TN 3.
☐	Chụp màn hình mạch và dán vào khung ảnh của file này	Chụp cả mạch lẫn nhãn ngõ vào/ngõ ra và giá trị đang hiển thị.

Phiên bản Logisim đang dùng	

Đọc màu dây và giá trị trong Logisim

Xanh lá sáng = 1 · **Xanh lá đậm** = 0 · **Xanh dương** = chưa xác định / chưa được điều khiển · **Đỏ** = lỗi/xung đột · **Cam** = hai đầu dây lệch số bit.

Ở mạch **tuần tự**, ngay sau khi mở file hoặc Reset Simulation, giá trị đang lưu có thể **chưa xác định** và hiển thị là **x** (hoặc dây xanh dương) cho đến khi mạch được ghi lần đầu. Đó là hành vi bình thường, không phải lỗi; nếu phiên bản Logisim của Sinh viên hiển thị khác, hãy ghi lại điều quan sát được.

Bước 0 – Kiểm tra nhanh. Trong circuit **Warmup**, đặt hai Input Pin 8 bit **A**, **B**, một **Adder** (Arithmetic, Data Bits = 8), một Output Pin 8 bit **S** và một Probe. **Dự đoán** S (viết hex) bằng tay trước khi Poke, rồi đọc S ở dạng thập phân **không dấu** và **có dấu** (bù 2).

A	B	S = A + B (hex)	S không dấu	S có dấu (bù 2)
0x20	0x0F			
0x7F	0x01			
0xFF	0x01			
0x64	0x64			

Circuit Warmup chỉ để làm quen, không nộp. Hãy để ý: cùng một dãy bit, hai cách đọc cho hai giá trị khác nhau; Thí nghiệm 1 sẽ trả lời "phần cứng báo cho phần mềm biết kết quả có đáng tin hay không" bằng cách nào.

THÍ NGHIỆM 1 — Bộ cộng/trừ 4 bit và các cờ trạng thái

Thời lượng gợi ý: 45 phút · Circuit: **FA, ADDSUB4** · Điểm: 2,0

> **Tình huống**
> Phép cộng là phép toán nền tảng của ALU: phép trừ, tăng/giảm và so sánh đều quy về bộ cộng. Sinh viên sẽ tự xây bộ cộng 4 bit từ cổng luận lý, biến nó thành bộ cộng/trừ, rồi tự thiết kế mạch **phát hiện tràn số** để biết khi nào kết quả 4 bit không còn đúng, thông tin mà CPU thật luôn cung cấp dưới dạng các cờ.

A. Bộ cộng toàn phần (FA) từ cổng luận lý

Bước 1.1 – Dự đoán. Cộng ba bit A + B + Cin cho kết quả từ 0 đến 3, viết được bằng hai bit (Cout S). Điền hai cột "dự đoán" của Bảng 1 bằng phép cộng số học, **chưa** dựng mạch.

DỰNG MẠCH **Bước 1.2.** Trong circuit **FA** (circuit mới), đặt 3 Input Pin **A, B, Cin** và 2 Output Pin **S, Cout**. Dựng theo slide: S = A ⊕B ⊕Cin; Cout = A·B + Cin·(A ⊕B), dùng chung cổng A ⊕B cho cả hai ngõ ra.

QUAN SÁT Dùng Poke tool (hoặc Analyze Circuit ▸ Table) để điền hai cột "đo".

Bảng 1. Bộ cộng toàn phần FA

A	B	Cin	S (dự đoán)	Cout (dự đoán)	S (đo)	Cout (đo)
0	0	0	0	0	0	0
0	0	1	1	0	1	0
0	1	0	0	1	1	0
0	1	1	1	1	0	1
1	0	0	1	0	1	0
1	0	1	0	1	0	1
1	1	0	0	1	0	1
1	1	1	1	0	1	1

Câu 1.1 (Phân tích). Cột Cout chính là hàm "có từ 2 ngõ vào trở lên bằng 1" mà slide Chương 4 dùng làm ví dụ thiết kế. Viết SoP tối giản của Cout (gom bìa Karnaugh 3 biến nếu cần) và tính C(B). So sánh với dạng Cout = A·B + Cin·(A ⊕B) khi cổng A ⊕B đã có sẵn để tạo S: tổng số cổng của cả mạch FA ở mỗi cách là bao nhiêu?

Trả lời:
Cách 1
Biểu thức SoP tối giản: Cout = AB + A·Cin + B·Cin -> C(B) = 3 + 2 + 2 + 2 = 9
S = A ⊕ B ⊕ Cin
Như vậy, mạch FA theo cách 1 sử dụng 6 cổng.
Cách 2:
Cout = A · B + Cin · (A ⊕ B)
S = A ⊕ B ⊕ Cin
Như vậy, mạch FA theo cách 2 sử dụng 5 cổng.

Câu 1.2 (Phân tích). Đếm **số tầng cổng** (độ sâu) của mạch FA ở bước 1.2 trên hai đường đi: (a) từ **Cin** đến **Cout**; (b) từ **A hoặc B** đến **Cout**. Ghi cách đếm.

Trả lời:
(a) Từ Cin đến Cout:
Cách đếm: Cin là ngõ vào trực tiếp của cổng AND (kết hợp với tín hiệu A ⊕ B), sau đó đi tiếp vào cổng OR để tạo ra Cout. (Tín hiệu Cin không đi qua cổng XOR).
Số tầng cổng: 2 tầng (1 cổng AND → 1 cổng OR).
(b) Từ A hoặc B đến Cout:
Cách đếm: Từ A hoặc B có hai đường đi đến Cout:
Nhánh 1: Đi qua 1 cổng AND (tính A · B) → 1 cổng OR (Độ sâu = 2 tầng).
Nhánh 2: Đi qua 1 cổng XOR (tính A ⊕ B) → 1 cổng AND (nhân với Cin) → 1 cổng OR (Độ sâu = 3 tầng).
Số tầng cổng (lấy đường đi dài nhất / đường tới hạn): 3 tầng (XOR →AND → OR).

B. Ghép thành bộ cộng 4 bit nối tiếp

DỰNG MẠCH **Bước 1.3.** Trong circuit **ADDSUB4**: Input Pin **A** (4 bit), **B** (4 bit), **SUB** (1 bit); Output Pin **S** (4 bit), **C** (1 bit). **Lần này chưa dùng SUB:** nối 4 mạch con FA liên tiếp (Cout của tầng i vào Cin của tầng i + 1; Cin của tầng 0 nối hằng 0). Dùng Splitter để tách A, B và gộp S. Đặt Probe (hoặc Output Pin tạm) cho ba dây nhớ **C1, C2, C3** giữa các tầng.

DỰ ĐOÁN Với ba phép cộng ở Bảng 2, dự đoán S, các bit nhớ và **số tầng mà bit nhớ đã lan qua** (bắt đầu đếm từ khi C1 = 1).

Bảng 2. Bit nhớ lan truyền trong bộ cộng 4 bit (Cin = 0)

A	B	S (nhị phân)	C1	C2	C3	Cout	Số tầng bit nhớ lan qua
0001	0001	0010	1	0	0	0	1
0111	0001	1000	1	1	1	0	3
1111	0001	0000	1	1	1	1	4

Câu 1.3 (Đánh giá). Với bộ cộng nối tiếp n bit, bit nhớ có thể phải đi qua tất cả các tầng. Dựa vào Câu 1.2, điền số tầng cổng của đường đi dài nhất (từ A[0]/B[0] đến Cout) ở Bảng 3 và rút ra công thức theo n. Nhận xét: vì sao CPU 32/64 bit thực tế không dùng thuần bộ cộng nối tiếp?

Bảng 3. Độ sâu của bộ cộng nối tiếp

n (số bit)	Số tầng FA mà bit nhớ đi qua	Số tầng cổng đến Cout
4	4	9
8	8	17
32	32	65
n	n	2n+1

Công thức, nhận xét:
Công thức:
-Số tầng FA bit nhớ đi qua: n
-Số tầng cổng đến Cout: 2n+1
Nhận xét: Bộ cộng nối tiếp có độ trễ lan truyền Cout tăng tuyến tính theo số bit (2n + 1 gate delays). Với CPU 32/64-bit, độ trễ này quá lớn gây giảm hiệu năng nghiêm trọng. Do đó, CPU thực tế dùng bộ cộng tính trước bit nhớ Carry Lookahead Adder (CLA) để xử lý song song và giảm độ sâu tầng cổng.

C. Biến thành bộ cộng/trừ

Câu 1.4 (Hiểu). A − B có thể viết là A + (−B), mà theo Chương 2, −B = B' + 1 (bù 2). Từ đó giải thích vì sao chỉ cần thêm 4 cổng XOR (mỗi cổng nhận B[i] và SUB) và nối SUB vào Cin của tầng 0 là bộ cộng thành bộ cộng/trừ. Điều gì xảy ra ở hai trường hợp SUB = 0 và SUB = 1?

Trả lời:
Phép trừ A - B = A + (-B) = A + B' + 1 (dạng bù 2). Cổng XOR hoạt động như một bộ đảo tín hiệu có điều khiển.
Khi SUB = 0: XOR giữ nguyên B[i] XOR 0 = B[i], đồng thời Cin = 0. Mạch thực hiện phép cộng A + B.
Khi SUB = 1: XOR đảo bit B[i] XOR 1 = B'[i] (bù 1), đồng thời Cin = 1 để cộng thêm 1. Mạch thực hiện A + B' + 1 (bù 2), tức là phép trừ A - B.
Từ cơ chế này, chỉ cần thêm các cổng XOR và nối tín hiệu SUB vào chân Cin là có thể dùng chung một mạch cộng để thực hiện cả phép cộng và phép trừ.

DỰNG MẠCH **Bước 1.4.** Thêm vào ADDSUB4: 4 cổng XOR cho B'[i] = B[i] ⊕SUB; nối SUB vào Cin của tầng 0. Kiểm tra nhanh: SUB = 0, A = 0011, B = 0100 → S = 0111; SUB = 1, A = 0111, B = 0011 → S = 0100.

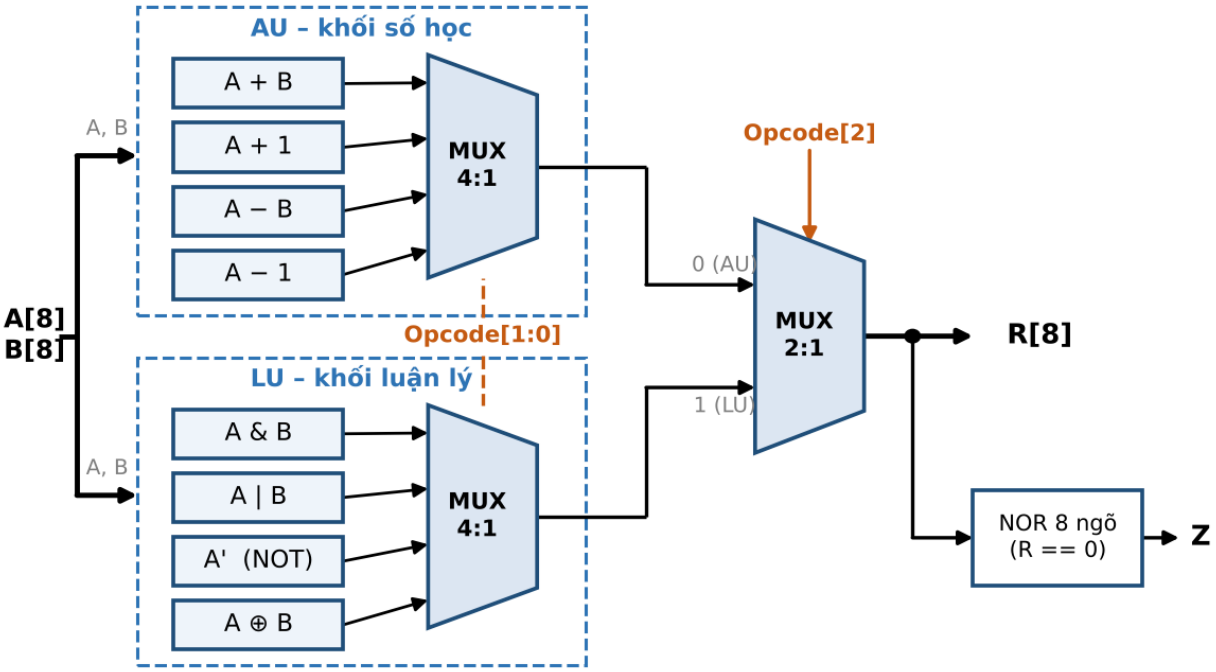
THÍ NGHIỆM 2 — ALU 8 bit và bộ so sánh: quy tắc nào luôn đúng?

Thời lượng gợi ý: 45 phút · Circuit: **ALU8, LT** · Điểm: 1,5

Tình huống

Một CPU chỉ cần **một** khối tính toán dùng chung cho nhiều phép toán: ALU. Sinh viên dựng ALU 8 bit theo bảng Opcode ở slide Chương 5 (Hình 1), rồi dùng chính ALU để **so sánh hai số** thay vì thiết kế bộ so sánh riêng, vì bảng chân trị của bộ so sánh hai số 8 bit có 2^16 hàng. Slide nêu quy tắc: "Y = A − B; Y[MSB] = 1 ⇒ A < B; Y[MSB] = 0 ⇒ A > B". Sinh viên sẽ **kiểm tra** quy tắc này có luôn đúng không.

A. Dựng ALU8 theo cấu trúc AU – LU – MUX



Hình 1. *Cấu trúc ALU 8 bit theo slide Chương 5. Opcode[1:0] chọn phép toán trong từng khối; Opcode[2] chọn khối AU hay LU.*

DỰNG MẠCH **Bước 2.1.** Trong circuit **ALU8**: Input Pin **A, B** (8 bit), **OP** (3 bit); Output Pin **R** (8 bit), **Z**. (a) **AU:** hai Adder (A + B, A + 1) và hai Subtractor (A − B, A − 1); hằng 1 lấy từ Constant 8 bit; chọn kết quả bằng Multiplexer 4:1 (8 bit) với Select = OP[1:0] (dùng Splitter tách OP). (b) **LU:** các cổng AND, OR, NOT, XOR có Data Bits = 8; chọn bằng một Multiplexer 4:1 khác với Select = OP[1:0]. (c) Multiplexer 2:1 (8 bit) chọn giữa AU và LU theo OP[2]. (d) **Z** = NOR của 8 bit R (Splitter + cổng NOR 8 ngõ vào).

Gợi ý: ngõ vào số k của Multiplexer được chọn khi Select = k. Hãy đối chiếu thứ tự nối với bảng Opcode ở slide (000: A + B, 001: A + 1, 010: A − B, 011: A − 1, 100: A & B, 101: A | B, 110: A', 111: A ⊕B).

DỰ ĐOÁN Điền cột "dự đoán" của Bảng 6 bằng tay (hex, 8 bit) **trước khi** Poke.

QUAN SÁT **Ca 1:** A = 0x3C, B = 0x0F. **Ca 2:** A = 0x00, B = 0x01. Điền cột "đo" và cờ Z của Ca 2.

Bảng 6. Kiểm thử ALU8 với 8 Opcode

Opcode	Phép toán	Ca 1: R dự đoán	Ca 1: R đo	Ca 2: R dự đoán	Ca 2: R đo	Ca 2: Z
000	A + B	0x4B	0x4B	0x01	0x01	0

Opcode	Phép toán	Ca 1: R dự đoán	Ca 1: R đo	Ca 2: R dự đoán	Ca 2: R đo	Ca 2: Z
001	A + 1	0x3D	0x3D	0x01	0x01	0
010	A − B	0x2D	0x2D	0xFF	0xFF	0
011	A − 1	0x3B	0x3B	0xFF	0xFF	0
100	A & B	0x3D	0x0C	0x01	0x00	1
101	A | B	0x3D	0x3F	0x01	0x01	0
110	A'	0xC3	0xC3	0xFF	0xFF	0
111	A ⊕B	0x00	0x33	0x01	0x01	0

Câu 2.1 (Phân tích). Ở Ca 2, phép nào cho R = 0xFF? Giá trị đó là 255 hay −1? ALU có "biết" điều đó không? Z = 1 ở phép nào, và Z = 1 có đồng nghĩa với "hai toán hạng bằng nhau" không?

Trả lời:
Ở Ca 2, các phép cho R = 0xFF là A − B, A − 1 và A'. Giá trị 0xFF có thể được hiểu là 255 nếu xét số không dấu, hoặc −1 nếu xét số có dấu theo biểu diễn bù 2. ALU chỉ xử lý các bit nên không tự biết giá trị đó là 255 hay −1.
Z = 1 ở phép A & B, vì kết quả R = 0x00. Z = 1 chỉ cho biết kết quả phép toán bằng 0, không có nghĩa là hai toán hạng bằng nhau.

Câu 2.2 (Áp dụng). ALU này không có phép "chép" (MOV: R = A). Đề xuất **hai** cách dùng đúng ALU này để có R = A (ghi Opcode và giá trị cần đặt cho B). Cách nào dùng được khi ta **chỉ chọn được thanh ghi** cho A và B, không đặt được hằng số? (Sẽ dùng ở Thí nghiệm 4.)

Trả lời:
Có thể dùng hai cách để có R = A:
Cách 1: Chọn Opcode 101 (A | B) và đặt B = 0x00. Khi đó A | 0 = A.
Cách 2: Chọn Opcode 111 (A ⊕ B) và đặt B = 0x00. Khi đó A ⊕ 0 = A.
Nếu chỉ chọn được thanh ghi cho A và B, không đặt được hằng số, thì cả hai cách đều dùng được nếu có một thanh ghi đang chứa giá trị 0x00 để chọn làm B.

B. So sánh hai số bằng ALU

DỰ ĐOÁN Với 6 cặp (A, B) ở Bảng 7, tính R = A − B trên **8 bit**, rồi điền cột "Quy tắc slide kết luận" (dùng <, =, >). Điền hai cột "Thực tế" theo hai cách hiểu: A, B là số **có dấu** (bù 2) và là số **không dấu**.

QUAN SÁT Đặt Opcode = 010 trên ALU8, ghi R, Z, R[7] của từng cặp. Cột **LT đo** để trống, sẽ điền sau Bước 2.3.

Bảng 7. So sánh bằng phép trừ

A	B	R = A − B (đo)	Z	R[7]	Quy tắc slide kết luận	Thực tế (có dấu)	Thực tế (không dấu)	LT đo
0x05	0x07	0xFE	0	1	<	<	<	1
0x09	0x09	0x00	1	0	=	=	=	0
0x64	0x9C	0xC8	0	1	<	>	<	0
0x9C	0x64	0x38	0	0	>	<	>	1
0xC8	0x0A	0xBE	0	1	<	<	>	1
0x0A	0xC8	0x42	0	0	>	>	<	0

Câu 2.3 (Phân tích). Tìm các dòng mà quy tắc slide cho kết luận **sai**. Với mỗi dòng sai, nêu nguyên nhân (gợi ý: phép trừ 8 bit có bị tràn không? A và B đang được hiểu là số có dấu hay không dấu?). Quy tắc ở slide chỉ đáng tin trong điều kiện nào?

Trả lời:
Quy tắc slide cho kết luận sai ở 4 dòng: (0x64, 0x9C), (0x9C, 0x64), (0xC8, 0x0A), (0x0A, 0xC8).
Với hai dòng (0x64, 0x9C) và (0x9C, 0x64), nếu xét số có dấu thì phép trừ bị tràn số, làm cho bit R[7] không phản ánh đúng dấu của kết quả thực. Vì vậy không thể chỉ dựa vào R[7] để so sánh số có dấu.
Với hai dòng (0xC8, 0x0A) và (0x0A, 0xC8), khi xét số không dấu, R[7] không phải là bit xác định quan hệ < hay > vì phép trừ có thể xảy ra hiện tượng mượn.
Quy tắc chỉ đúng khi phép trừ không xảy ra hiện tượng tràn số (Overflow = 0), hoặc khi hai số A và B cùng dấu. Khi có overflow hoặc khi xét số không dấu, cần dùng thêm cờ phù hợp để kết luận chính xác.

THIẾT KẾ **Bước 2.2 – Sửa quy tắc cho số có dấu.** Thiết kế mạch **LT** (A < B, số có dấu) chỉ dựa vào ba bit A[7], B[7], R[7] (R = A − B). Gợi ý: nếu A và B cùng dấu thì phép trừ không bao giờ tràn nên LT = R[7]; nếu khác dấu thì LT phụ thuộc dấu của A. Điền Bảng 8, bìa Karnaugh, rồi viết biểu thức tối giản.

Bảng 8. Bảng chân trị của LT(A7, B7, R7)

A7	B7	R7	LT (A < B có dấu)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1

A7	B7	R7	LT (A < B có dấu)
1	1	0	0
1	1	1	1

A7 \ B7 R7	00	01	11	10
0	0	1	0	0
1	1	1	1	0

LT tối giản = …	B7'·R7 + A7·B7' + A7·R7
Số cổng cần dùng (kể cả cổng NOT)	5

Câu 2.4 (Phân tích). Biểu thức LT quen thuộc không? (Gợi ý: so sánh với hàm đa số ở slide Chương 4 và với Cout của FA ở Thí nghiệm 1, trong đó một ngõ vào được lấy bù.) Giải thích bằng lời vì sao có sự giống nhau đó (nhớ lại A − B = A + B' + 1).

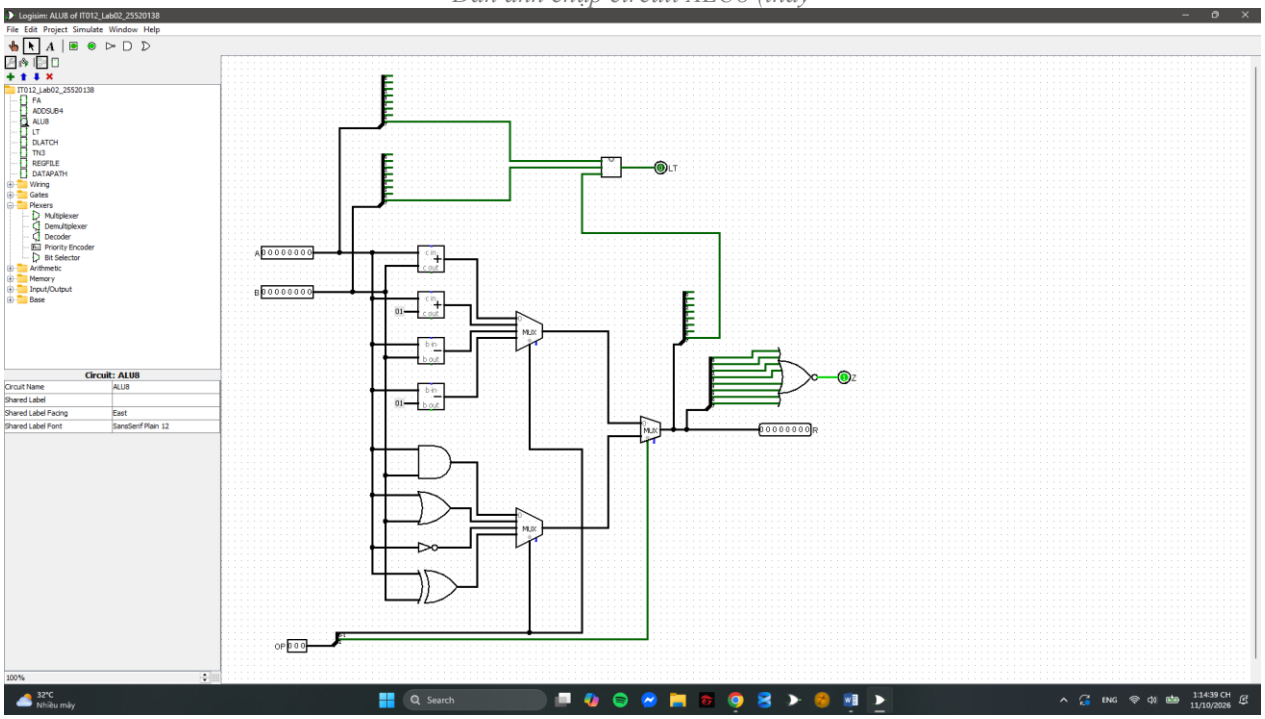
Trả lời:
Biểu thức: B7'·R7 + A7·B7' + A7·R7 chính là hàm đa số của 3 biến, có cấu trúc tương tự như ngõ ra truyền nhớ Cout của bộ cộng toàn phần (FA) ở Thí nghiệm 1 nhưng thay ngõ vào B bằng B'.
Giải thích: Phép trừ trong ALU thực chất là phép cộng bù hai (A - B = A + B' + 1). Do đó, các biến tham gia quyết định dấu và quan hệ lớn/nhỏ gồm dấu của A (A7), dấu đã đảo của B (B7'), và dấu của kết quả phép trừ (R7). Điều kiện để xác định A < B (có dấu) dựa trên sự áp đảo giá trị giữa ba tín hiệu này, dẫn đến hàm logic có dạng hàm đa số y hệt như mạch tạo nhớ Cout.

DỰNG MẠCH **Bước 2.3.** Dựng mạch con **LT** (ngõ vào A7, B7, R7; ngõ ra LT), nối vào ALU8 (A[7], B[7], R[7]) và điền cột **LT đo** của Bảng 7. LT có sửa được các dòng sai ở Câu 2.3 không?

Câu 2.5 (Đánh giá – mở rộng). Với số **không dấu**, A < B khi nào? (Gợi ý: ý nghĩa của cờ C khi trừ ở Câu 1.6b.) Kiểm tra điều kiện của Sinh viên trên hai dòng 0xC8 và 0x0A của Bảng 7 bằng cách tính A − B trên 9 bit. Sau đó, so sánh chi phí: bộ so sánh trực tiếp hai số 8 bit cần bảng chân trị bao nhiêu hàng, so với "ALU + LT"? Khi nào vẫn nên dùng bộ so sánh riêng?

Trả lời:
- Điều kiện: A < B khi phép trừ A - B có mượn, tương ứng với cờ mượn (Borrow) C = 1 (hoặc C = 0 tùy theo quy ước cờ nhớ/mượn của ALU khi thực hiện A + B' + 1).
- Kiểm tra trên hai dòng 0xC8 và 0x0A (tính trên 9 bit):
A = 0xC8 = 0 1100 1000_2 (200 hệ 10)
B = 0x0A = 0 0000 1010_2 (10 hệ 10)
Phép trừ 9-bit: 200 - 10 = 190 > 0 (không có mượn âm ra ngoài), do đó A >= B (phù hợp với thực tế không dấu).
- So sánh chi phí phần cứng:
+ Bộ so sánh trực tiếp hai số 8-bit cần bảng chân trị với 2^{16} = 65536 hàng.
+ Tổ hợp "ALU + LT" tận dụng khối ALU có sẵn và chỉ bổ sung thêm một mạch logic rất nhỏ cho LT, tiết kiệm đáng kể tài nguyên phần cứng.
- Khi nào nên dùng bộ so sánh riêng: Khi hệ thống chỉ thuần túy cần chức năng so sánh độ lớn mà không cần các phép toán số học/logic khác của ALU, hoặc khi cần tối ưu hóa tốc độ cao (độ trễ truyền thấp) cho riêng khối so sánh.

Dán ảnh chụp circuit ALU8 (thấy



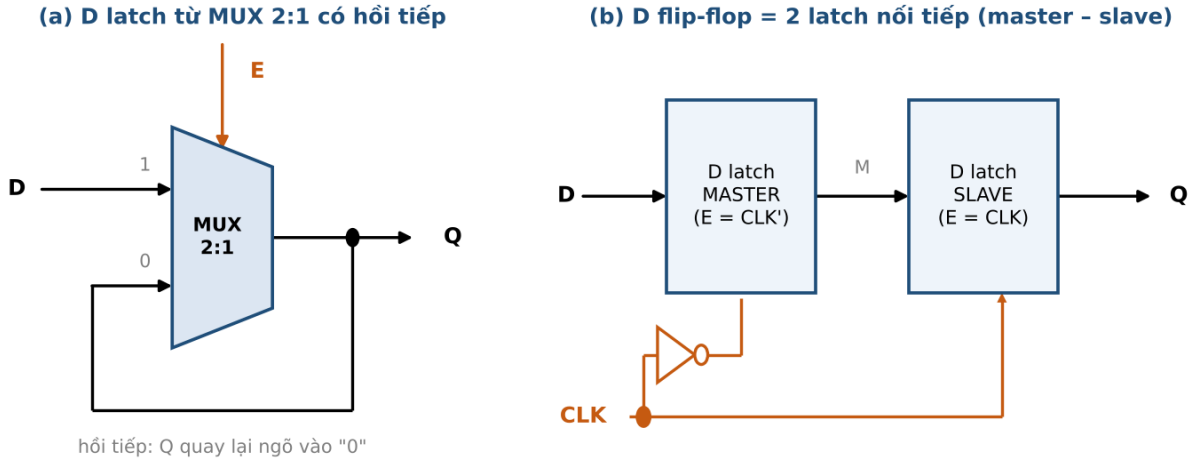
rõ A, B, OP, R, Z) và mạch con LT

THÍ NGHIỆM 3 — Mạch tuần tự: D latch và D flip-flop

Thời lượng gợi ý: 45 phút · Circuit: **TN3 (mạch con: DLATCH, DFF_MS)** · Điểm: 3,0

Tình huống

ALU ở Thí nghiệm 2 là mạch tổ hợp: ngõ ra đổi ngay khi ngõ vào đổi và **không nhớ** gì. Muốn CPU giữ lại kết quả để dùng cho lệnh sau, cần **thiết bị lưu trữ** (slide Chương 4). Sinh viên sẽ tự dựng D latch từ bảng trạng thái ở slide, so sánh nó với D flip-flop có sẵn, rồi ghép hai latch thành một flip-flop và tìm hiểu vì sao thanh ghi phải dùng flip-flop.



Hình 2. *(a) D latch dựng từ MUX 2:1 có hồi tiếp; (b) D flip-flop ghép từ hai latch (master – slave).*

A. D latch dựng từ bảng trạng thái

Slide Chương 4 mô tả D latch bằng bảng trạng thái kế tiếp: **Q+ = Q khi E = 0** (giữ), **Q+ = D khi E = 1** (nhận). Đó chính là một **MUX 2:1** mà ngõ vào "giữ" được nối với **ngõ ra Q của chính nó** (Hình 2a).

DỰNG MẠCH **Bước 3.1.** Trong circuit **DLATCH**: Input Pin **D, E**; Output Pin **Q**; một Multiplexer 1 bit (Select = E, ngõ vào 1 = D, ngõ vào 0 = **Q nối ngược về**).

DỰ ĐOÁN Chọn **Simulate ▸ Reset Simulation**, rồi thực hiện lần lượt các thao tác ở Bảng 9 (đổi E và D bằng Poke). Điền Q dự đoán **trước**; nếu Q chưa xác định thì ghi **x**.

QUAN SÁT Sau đó điền cột "Q đo".

Bảng 9. Chuỗi thao tác trên DLATCH

Bước	E	D	Q dự đoán	Q đo
1	0	0	x	x
2	0	1	x	x
3	1	1	1	1
4	0	1	1	1
5	0	0	1	1
6	1	0	0	0
7	1	1	1	1
8	1	0	0	0
9	0	0	0	0

Bước	E	D	Q dự đoán	Q đo
10	0	1	0	0

Câu 3.1 (Hiểu). Vì sao ở bước 1–2 Q chưa xác định dù đã có D? Điều đó nói gì về việc phải **khởi tạo** (reset) thiết bị lưu trữ trong mạch thật?

Trả lời:
Lý do Q chưa xác định ở bước 1 và 2: Khi E = 0, bộ Mux giữ ngõ ra ở trạng thái duy trì và không nhận dữ liệu mới từ D. Do hệ thống vừa khởi động và chưa từng thực hiện thao tác ghi dữ liệu nào trước đó, đường phản hồi chưa có mức logic cố định, dẫn đến ngõ ra Q nhận giá trị bất định (x).
Ý nghĩa của việc khởi tạo (Reset) trong mạch thật: Trong thực tế, các linh kiện lưu trữ luôn rơi vào trạng thái ngẫu nhiên lúc vừa bật nguồn. Vì vậy, việc thiết kế chân Reset là bắt buộc để đưa mạch về trạng thái ban đầu xác định (thường là 0), giúp toàn bộ hệ thống vận hành đúng ngay từ đầu và tránh lỗi dữ liệu rác.

Câu 3.2 (Phân tích). Slide Chương 4 định nghĩa mạch tổ hợp là mạch **không có hồi tiếp**; DLATCH có đường hồi tiếp từ Q về MUX. Khi E = 0, mô tả đường đi của tín hiệu để giải thích vì sao hồi tiếp làm mạch "nhớ". Khi E = 1 thì hồi tiếp còn tác dụng không?

Trả lời:
Đường đi tín hiệu khi E = 0: Lúc E = 0, Mux chuyển sang chọn kênh 0, cho phép tín hiệu từ ngõ ra Q quay ngược trở lại ngõ vào của chính nó. Mạch tạo thành một vòng lặp kín tự nuôi, giúp giá trị logic tại Q được giữ nguyên liên tục theo thời gian. Đây chính là cơ chế tạo nên khả năng "nhớ" của mạch.
Tác dụng của hồi tiếp khi E = 1: Khi E = 1, Mux ngắt đường hồi tiếp và đổi sang nhận tín hiệu từ cổng D. Lúc này vòng lặp bị tạm dừng tác dụng, ngõ ra Q sẽ lập tức thay đổi và cập nhật liên tục theo giá trị mới của ngõ vào D.

B. Latch và flip-flop: cùng ngõ vào, khác kết quả

DỰNG MẠCH **Bước 3.2.** Trong circuit **TN3**: Input Pin **D** và **CLK** (đổi bằng Poke, **không** dùng Clock tự chạy); một DLATCH với E = CLK (ngõ ra **Q1**); một **D Flip-Flop** (Memory, kích cạnh lên – Rising Edge) với ngõ vào D, CLK (ngõ ra **Q2**).

DỰ ĐOÁN Bảng 10 cho CLK và D theo từng **khe thời gian** (mỗi cột một khe; tại ranh giới giữa hai khe chỉ đổi **một** tín hiệu: CLK hoặc D, không bao giờ cả hai). Giả sử ban đầu Q1 = Q2 = 0. Điền hàng dự đoán Q1, Q2. Đây là bài "hoàn thành dạng sóng" ở slide Chương 4.

QUAN SÁT Đưa mạch về Q1 = Q2 = 0 (Reset Simulation; D = 0; CLK 0 → 1 → 0). Sau đó Poke từng khe theo bảng và ghi Q1, Q2 đo được. Hàng Q3 để trống đến Bước 3.3.

Bảng 10. Dạng sóng theo khe thời gian

Khe	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
CLK	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
D	0	1	1	0	0	1	1	1	1	0	0	1	1	0	0	0
Q1 dự đoán	0	0	0	1	0	1	0	0	1	1	1	0	0	1	1	0
Q1 đo	0	0	1	0	0	0	1	1	1	1	0	1	1	1	0	0
Q2 dự đoán	1	0	0	1	1	0	1	1	0	0	1	0	0	1	1	0
Q2 đo	0	0	1	1	1	1	1	1	1	1	0	0	0	0	0	0
Q3 đo (DFF_MS)																

Câu 3.3 (Phân tích). Liệt kê các khe mà Q1 ≠ Q2. Với từng khe, giải thích bằng định nghĩa "tích cực theo **mức**" (latch) và "tích cực theo **cạnh**" (flip-flop) ở slide. Trong khoảng CLK = 1, D đổi một lần và Q1 đổi theo, còn Q2 thì không: vì sao?

Trả lời:
Các khe thời gian mà Q1 khác Q2 gồm các khe 3, 4, 5 và 11, 12, 13.
Giải thích:
Tại các khe 3, 4 và 5:
- Q1 là Latch hoạt động theo mức: Trong giai đoạn xung CLK duy trì ở mức cao, mạch hoàn toàn "mở", khiến tín hiệu ngõ ra liên tục bám sát và thay đổi theo mọi biến động của ngõ vào D.
- Q2 là Flip-Flop kích hoạt theo sườn xung: Thiết bị chỉ chốt dữ liệu tại đúng khoảnh khắc chuyển tiếp sườn lên của CLK. Trong suốt thời gian CLK giữ nguyên mức hoặc đi xuống, Q2 đóng hoàn toàn và giữ nguyên trạng thái trước đó, dẫn đến hiện tượng sai lệch tín hiệu giữa Q1 và Q2 trải dài qua các giai đoạn 3, 4 đến hết 5.
Tại các khe 11, 12 và 13:
- Tương tự, sự biến đổi của tín hiệu D khi CLK đang ở mức cao tiếp tục đi thẳng qua Latch làm Q1 biến đổi liên tục.
- Trái lại, Flip-Flop bỏ qua các chuyển mạch ngõ vào diễn ra giữa chu kỳ vì không có sườn lên mới, buộc Q2 duy trì giá trị cũ, làm hai đầu ra lệch nhau xuyên suốt từ khe 11, 12 tới 13.
Nguyên do trong khoảng CLK = 1, D đổi 1 lần mà Q1 đổi theo còn Q2 thì không:
- Với Q1 (Latch): Do cơ chế nhạy theo mức (Level-triggered), thời gian CLK = 1 đóng vai trò như một cửa sổ cho phép tín hiệu truyền trực tiếp, nên bất kỳ sự thay đổi nào của D cũng lập tức phản ánh lên Q1.
- Với Q2 (Flip-Flop): Do cơ chế nhạy theo sườn (Edge-triggered), thiết bị chỉ "chụp" giá trị D đúng vào thời điểm CLK chuyển từ 0 lên 1. Sau thời điểm đó, dù CLK vẫn ở mức 1 thì mạch đã khóa ngõ vào, khiến biến động của D không thể tác động tới Q2.

C. Vì sao thanh ghi dùng flip-flop mà không dùng latch?

DỰNG MẠCH **Bước 3.4.** Trong circuit TN3, dựng hai chuỗi hai tầng: **(i)** hai DLATCH nối tiếp (Q của latch 1 vào D của latch 2), cả hai có E = CLK; **(ii)** hai D Flip-Flop nối tiếp cùng CLK. Cả hai chuỗi dùng chung ngõ vào D.

DỰ ĐOÁN Ban đầu mọi Q = 0. Mỗi **xung** là CLK: 0 → 1 → 0, D giữ nguyên trong xung. Điền Q1, Q2 (viết dạng "Q1 Q2") sau mỗi xung, rồi đo.

Bảng 12. Chuỗi hai tầng: latch và flip-flop

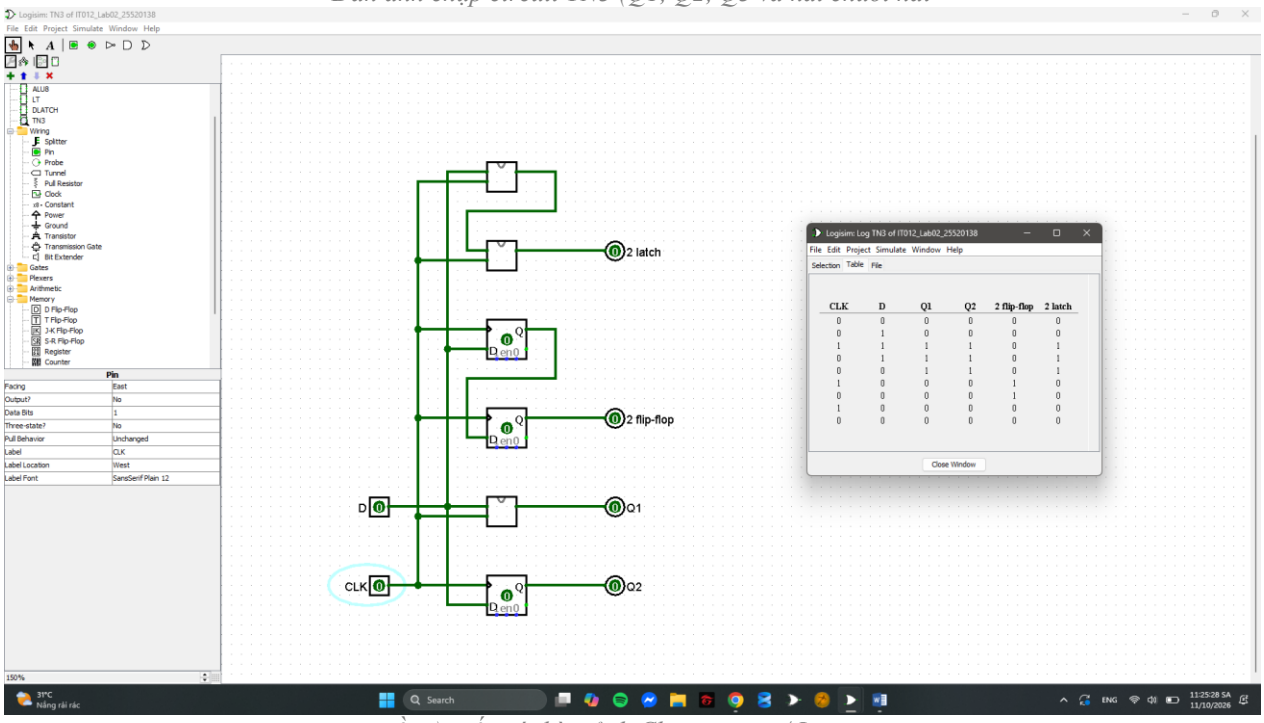
Xung	D	2 latch: dự đoán	2 latch: đo	2 flip-flop: dự đoán	2 flip-flop: đo
1	1	1 1	1 1	1 0	1 0
2	0	1 0	0 0	0 1	0 1
3	0	0 1	0 0	1 0	0 0

Câu 3.5 (Đánh giá). Từ Bảng 12, dữ liệu đi qua mấy tầng trong **một** xung ở mỗi cấu hình? Từ đó dự đoán điều gì xảy ra với lệnh **R1 ← R1 + R0** (ngõ ra ALU quay lại ngõ vào chính thanh ghi R1) khi R1 được làm bằng latch trong suốt lúc CLK = 1, và khi R1 làm bằng flip-flop. (Sinh viên sẽ kiểm chứng ở Thí nghiệm 4.)

Trả lời:
Cấu hình 2 Latch: Dữ liệu dịch chuyển qua 2 tầng.
Giải thích: Latch hoạt động theo mức (Level-triggered). Trong toàn bộ khoảng thời gian xung CLK ở mức cao (CLK = 1), kênh truyền dữ liệu qua cả 2 Latch được mở đồng thời, cho phép thông tin đi xuyên suốt qua cả hai tầng liên tiếp chỉ trong một chu kỳ nhịp.
Cấu hình 2 Flip-Flop: Dữ liệu chỉ truyền qua 1 tầng.
Giải thích: Flip-Flop hoạt động theo cạnh (Edge-triggered), chỉ chốt thông tin tại thời điểm chuyển mạch sườn lên của CLK. Do đó, dữ liệu sau khi qua tầng 1 phải dừng lại và chờ tới khi có sườn lên của chu kỳ xung nhịp tiếp theo mới tiếp tục truyền sang tầng 2.

Dự đoán với lệnh R1 ← R1 + R0:
Trường hợp R1 được cấu tạo từ Latch:
Hiện tượng: Giá trị lưu trong R1 bị biến đổi liên tục, rơi vào trạng thái không ổn định và tạo ra kết quả sai lệch nghiêm trọng.
Giải thích: Do Latch liên tục mở trong khoảng thời gian CLK = 1, kết quả tính toán từ đầu ra của khối ALU lập tức quay ngược trở lại đầu vào của thanh ghi R1. Quá trình này lặp đi lặp lại tạo thành một vòng lặp phản hồi khép kín (hiện tượng đua dữ liệu - Race condition), làm dữ liệu cộng dồn vô hạn ngay trong cùng một xung nhịp.
Trường hợp R1 được cấu tạo từ Flip-Flop:
Hiện tượng: Mạch vận hành chính xác, dữ liệu cập nhật ổn định đúng theo thiết kế.
Giải thích: Flip-Flop tạo ra cơ chế cách ly giữa đầu vào và đầu ra. Tín hiệu kết quả từ ALU chỉ được chốt vào R1 đúng một lần duy nhất tại khoảnh khắc sườn lên của CLK, nhờ đó triệt tiêu hoàn toàn vòng lặp phản hồi không kiểm soát.

Dán ảnh chụp circuit TN3 (Q1, Q2, Q3 và hai chuỗi hai

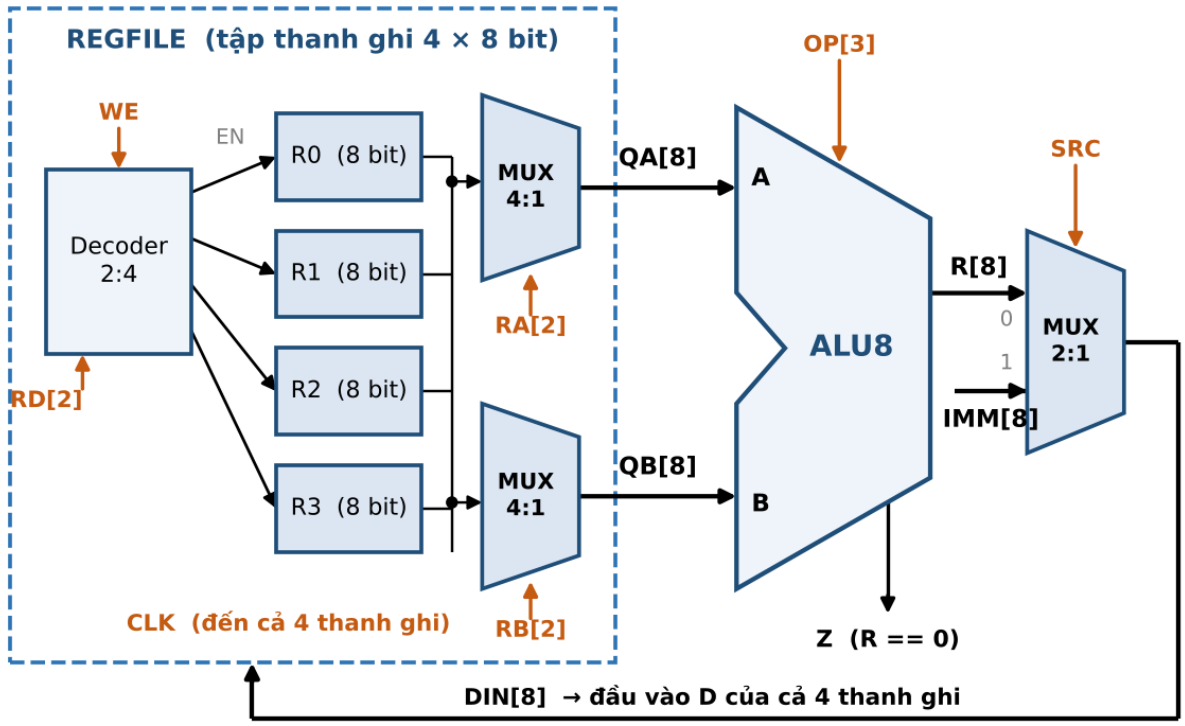


tầng); nếu có, kèm ảnh Chronogram / Logging

THÍ NGHIỆM 4 — Tập thanh ghi và đường dữ liệu 8 bit

Thời lượng gợi ý: 60 phút · Circuit: **REGFILE, DATAPATH** · Điểm: 3,0

> **Tình huống**
> Slide Chương 5 minh hoạ lệnh **C = A + B**: đưa A, B từ bộ nhớ vào **tập thanh ghi**, **ALU** tính, kết quả ghi lại vào tập thanh ghi. Sinh viên ghép các khối đã dựng thành **đường dữ liệu** của một CPU thu nhỏ và điều khiển nó bằng tay: mỗi "lệnh" là một bộ tín hiệu điều khiển cộng với một xung CLK.



Hình 3. *Đường dữ liệu mini: REGFILE (decoder 2:4 + 4 thanh ghi + 2 MUX đọc) → ALU8 → MUX chọn nguồn ghi (SRC).*

A. Tập thanh ghi 4 × 8 bit

Slide Chương 5 vẽ tập thanh ghi với một cổng đọc. ALU cần **hai toán hạng cùng lúc** nên REGFILE ở đây có **hai cổng đọc** (RA, RB) và **một cổng ghi** (RD).

DỰNG MẠCH **Bước 4.1.** Trong circuit **REGFILE**: Input Pin **CLK, WE** (1 bit), **RD** (2 bit, địa chỉ ghi), **DIN** (8 bit), **RA, RB** (2 bit, địa chỉ đọc); Output Pin **QA, QB** (8 bit) và **R0 … R3** (8 bit, để theo dõi). Dùng: một **Decoder 2:4** (Select = RD; nối WE vào ngõ Enable nếu có, hoặc dùng 4 cổng AND với WE), **4 Register** 8 bit (D = DIN, En = ngõ ra thứ k của decoder, Clock = CLK) và **2 Multiplexer 4:1** 8 bit chọn Rk theo RA và theo RB.

DỰ ĐOÁN Với các bước 5–8 của Bảng 13, điền cột "R1 dự đoán" **trước** khi Poke. Mỗi bước kết thúc bằng một xung CLK (0 → 1 → 0) trừ khi ghi chú khác.

QUAN SÁT Ghi R0 … R3 (hex) sau từng bước.

Bảng 13. Ghi và đọc trên REGFILE

Bước	Thao tác	R1 dự đoán	R0	R1	R2	R3
0	Vừa Reset Simulation (WE = 0)	0x00	0x00	0x00	0x00	0x00
1	RD = 0, WE = 1, DIN = 0x11, xung CLK	–	0x11	0x00	0x00	0x00
2	RD = 1, WE = 1, DIN = 0x22, xung CLK	–	0x11	0x22	0x00	0x00
3	RD = 2, WE = 1, DIN = 0x33, xung CLK	–	0x11	0x22	0x33	0x00
4	RD = 3, WE = 1, DIN = 0x44, xung CLK	–	0x11	0x22	0x33	0x44
5	RD = 1, **WE = 0**, DIN = 0x99, xung CLK	0x22	0x11	0x22	0x33	0x44
6	RD = 1, WE = 1, DIN = 0x99, **CLK vẫn = 0**	0x33	0x11	0x22	0x33	0x44
7	CLK: 0 → 1 (giữ CLK = 1)	0x99	0x11	0x99	0x33	0x44
8	CLK: 1 → 0	0x99	0x11	0x99	0x33	0x44

Câu 4.1 (Hiểu). Vì sao cần tín hiệu WE ngoài CLK? Điều gì xảy ra nếu bỏ WE (mỗi xung CLK đều ghi DIN vào thanh ghi do RD chọn) khi ta chỉ muốn **so sánh** hai thanh ghi mà không muốn ghi kết quả?

Trả lời:
Cần tín hiệu WE (Write Enable) kết hợp với xung CLK để kiểm soát chính xác thời điểm nào được phép ghi dữ liệu vào tập thanh ghi, tránh việc thanh ghi bị ghi đè dữ liệu ngoài ý muốn mỗi khi có xung nhịp CLK.
Nếu bỏ tín hiệu WE (nghĩa là mọi xung CLK đều thực hiện ghi), thì trong các tác vụ chỉ dùng để đọc, trích xuất dữ liệu hoặc so sánh hai thanh ghi (ví dụ: ALU nhận dữ liệu từ QA và QB để so sánh), mỗi khi có chu kỳ xung nhịp đi qua, dữ liệu ngõ vào DIN sẽ vô tình bị ghi đè lên thanh ghi đang được chọn bởi RD. Điều này làm mất dữ liệu cũ, sai lệch trạng thái lưu trữ của hệ thống và gây lỗi chương trình.

Câu 4.2 (Phân tích). Khi RD = RA = 2, WE = 1 và DIN thay đổi: QA đổi **ngay khi DIN đổi** hay **chỉ sau cạnh lên**? Vì sao điều này quan trọng cho lệnh R2 ← R2 + R1 (liên hệ Bảng 12)?

Trả lời:
Thời điểm biến đổi của QA:
- QA chỉ cập nhật sau sườn lên (cạnh lên) của xung CLK khi WE = 1, chứ không thay đổi ngay lập tức lúc DIN biến đổi.
- Cách kiểm tra: Trên phần mềm Logisim, quan sát đầu ra QA chỉ đảo giá trị tại khoảnh khắc xung CLK chuyển biến từ mức 0 lên mức 1.
Tầm quan trọng đối với thao tác lệnh R2 ← R2 + R1:
- Việc cập nhật theo sườn lên giúp bảo toàn giá trị gốc của R2 ở đầu vào QA trong suốt thời gian ALU tính toán, ngăn chặn hiện tượng dữ liệu mới từ DIN ghi đè làm sai lệch toán hạng đầu vào.
- Cách kiểm tra: Khối ALU hoàn tất phép cộng với dữ liệu R2 ban đầu trước, sau đó kết quả mới chính thức được lưu vào thanh ghi ở chu kỳ xung nhịp kế tiếp.

B. Ghép đường dữ liệu

DỰNG MẠCH **Bước 4.2.** Trong circuit **DATAPATH**: dùng mạch con REGFILE, mạch con ALU8 và một Multiplexer 2:1 (8 bit) chọn DIN: **SRC = 0 → R** (kết quả ALU), **SRC = 1 → IMM** (hằng số). Input Pin: **IMM** (8 bit), **SRC**, **OP** (3 bit), **RA, RB, RD** (2 bit), **WE**, **CLK**; Output Pin: **R0 … R3**, **R**, **Z**. Nối theo Hình 3 (A ← QA, B ← QB).

> **Quy ước "một lệnh"**
> - Một lệnh = đặt đúng các tín hiệu điều khiển (SRC, OP, RA, RB, RD, WE, IMM) **rồi** tạo một xung CLK (0 → 1 → 0).
> - Lệnh **nạp hằng**: SRC = 1. Lệnh **ALU**: SRC = 0. Lệnh **chỉ kiểm tra** (không ghi): WE = 0, đọc cờ Z.
> - Ô không dùng đến ghi dấu "–".

DỰ ĐOÁN Điền **từ vựng điều khiển** của 5 lệnh ở Bảng 14 (số thanh ghi viết dạng thập phân, OP dạng nhị phân, IMM dạng hex).

Bảng 14. Tín hiệu điều khiển của từng lệnh

Lệnh	SRC	OP	RA	RB	RD	WE	IMM
(1) R0 ← 0x05	1				0	1	0x05
(2) R1 ← 0x07	1				1	1	0x07
(3) R2 ← R0 + R1 (C = A + B ở slide)	0	000	0	1	2	1	
(4) R3 ← R2 − R0	0	010	2	0	3	1	
(5) So sánh R3 với R1 (không ghi)	0	010	3	1		0	

QUAN SÁT Chạy lần lượt 5 lệnh (từ trạng thái Reset Simulation) và ghi R0 … R3 (hex) và cờ Z sau mỗi lệnh.

Bảng 15. Kết quả chạy chương trình (1)–(5)

Sau lệnh	R0	R1	R2	R3	Z
(1)	0x05	0x00	0x00	0x00	–
(2)	0x05	0x07	0x00	0x00	–
(3)	0x05	0x07	0x0C	0x00	0
(4)	0x05	0x07	0x0C	0x07	0
(5)	0x05	0x07	0x0C	0x07	1

Câu 4.3 (Hiểu). Lệnh (5) không ghi gì nhưng vẫn hữu ích. Z = 1 cho biết điều gì? So với quy tắc "MSB" ở Thí nghiệm 2, kết luận "bằng nhau" từ Z có luôn đúng không? Vì sao?

Trả lời:
Ý nghĩa của Z = 1: Cờ Z = 1 cho biết kết quả phép trừ của ALU bằng 0, nghĩa là hai toán hạng được đem ra so sánh có giá trị bằng nhau.
Tính đúng đắn của kết luận "bằng nhau" từ cờ Z so với quy tắc MSB: Không phải lúc nào cũng đúng đối với trường hợp số có dấu. Cờ Z chỉ xác định toàn bộ các bit kết quả bằng 0 mà không xét đến dấu và hiện tượng tràn số (overflow), do đó việc kết luận hai số có dấu bằng nhau chỉ dựa vào cờ Z là chưa đủ chính xác trong mọi trường hợp.

TỔNG KẾT VÀ NỘP BÀI

Danh sách kiểm tra trước khi nộp

☑	Nội dung
☑	File **IT012_Lab02_<MSSV>.circ** có đủ các circuit **FA, ADDSUB4, ALU8, LT, DLATCH, TN3, REGFILE, DATAPATH**; mọi Input/Output Pin đều có nhãn.
☑	File báo cáo này đã điền đầy đủ các bảng, câu hỏi và dán ảnh chụp mạch vào các khung nét đứt.
☑	Số liệu trong báo cáo khớp với mạch nộp; sau khi sửa mạch con (FA, ALU8…) đã kiểm tra lại các mạch dùng nó.
☑	Đặt tên file báo cáo: **IT012_Lab02_<MSSV>_BaoCao**; nộp cả hai file theo hướng dẫn của giảng viên.

Phản tư

Câu R1. Từ Thí nghiệm 1 và 2, Sinh viên rút ra điều gì về sự khác nhau giữa "kết quả **đúng về mặt bit**" và "kết quả **đúng về mặt ý nghĩa**" (có dấu / không dấu)? Phần cứng giúp phần mềm phân biệt hai điều đó bằng cách nào?

Trả lời:
Sự khác nhau: "Đúng về mặt bit" là phần cứng thực hiện đúng phép toán nhị phân trên các chuỗi bit; còn "đúng về mặt ý nghĩa" phụ thuộc vào cách quy ước diễn giải chuỗi bit đó là số có dấu (bù 2) hay số không dấu (ví dụ: 0xFF là 255 nếu không dấu, nhưng là -1nếu có dấu).
Phần cứng phân biệt: Phần cứng không tự phân biệt ý nghĩa dữ liệu mà chỉ tính toán thuần túy. Việc phân biệt hoàn toàn do phần mềm/người lập trình quy ước ngữ cảnh sử dụng.

Câu R2. Từ Thí nghiệm 3 và 4: trong DATAPATH, khối nào là mạch **tổ hợp**, khối nào là mạch **tuần tự**? Chỉ ra một vòng hồi tiếp (từ ngõ ra thanh ghi, qua ALU, quay về ngõ vào thanh ghi) và giải thích vì sao vòng này không gây hỗn loạn.

Trả lời:
Phân loại khối:
- Mạch tổ hợp: ALU, MUX, các cổng logic (ngõ ra phụ thuộc ngay lập tức vào ngõ vào hiện tại).
- Mạch tuần tự: Tap thanh ghi REGFILE (lưu trữ trạng thái và chỉ cập nhật theo xung CLK).
Vòng hồi tiếp & Giải thích không gây hỗn loạn:
-Vòng hồi tiếp: Từ ngõ ra thanh ghi ➔ qua ALU ➔ qua MUX ➔ quay lại ngõ vào DIN của thanh ghi.
Lý do không hỗn loạn: Thanh ghi sử dụng Flip-Flop kích theo cạnh xung CLK (0 ➔ 1). Dữ liệu từ ALU chỉ được chốt vào thanh ghi tại thời điểm cạnh xung xuất hiện chứ không truyền liên tục, giúp tránh vòng lặp vô tận.

Câu R3. Nêu **một** lỗi hoặc nhầm lẫn Sinh viên đã gặp trong buổi thực hành, Sinh viên phát hiện ra bằng cách nào và bài học rút ra là gì?

Trả lời:
Lỗi gặp phải: Đấu nối nhầm giữa ngõ ra ALU, ngõ ra MUX và Pin R gây chập tín hiệu (báo lỗi đường dây màu đỏ E) hoặc sai lệch kích thước bit giữa các khối (báo lỗi đường dây màu cam Incompatible widths).
Cách phát hiện: Quan sát màu sắc đường dây mô phỏng và bảng báo lỗi của Logisim khi kích xung CLK nhưng kết quả thanh ghi không đổi.
Bài học rút ra: Cần kiểm tra kỹ kích thước Data Bits của từng thuộc tính linh kiện và phân tách rõ luồng đường đi ngõ ra trước khi chạy mô phỏng.

Câu R4 (mở, không chấm). Để DATAPATH chạy được một chương trình có vòng lặp hay rẽ nhánh ("nếu Z = 1 thì làm A, ngược lại làm B") mà **không cần người đặt tín hiệu bằng tay**, Sinh viên cần thêm những khối nào? Ghi ý tưởng, chưa cần vẽ mạch.

Trả lời:
Để DATAPATH có thể tự chạy chương trình có vòng lặp hoặc rẽ nhánh, cần bổ sung một số khối điều khiển. Trước hết cần có bộ đếm chương trình để lưu địa chỉ lệnh hiện tại và cập nhật sang lệnh tiếp theo. Cần có bộ nhớ lệnh để lưu chương trình và mạch điều khiển để giải mã lệnh, tạo ra các tín hiệu điều khiển cho DATAPATH.
Ngoài ra, cần có mạch chọn địa chỉ lệnh tiếp theo, sử dụng các cờ như Z để quyết định rẽ nhánh. Khi Z = 1 có thể chọn địa chỉ nhánh, còn khi Z = 0 tiếp tục thực hiện lệnh kế tiếp. Nhờ đó, CPU có thể tự thực hiện các câu lệnh tuần tự, rẽ nhánh và vòng lặp mà không cần đặt tín hiệu bằng tay.

Tài liệu tham khảo

- Slide bài giảng Chương 4 – Mạch số và Chương 5 – Ứng dụng mạch số (IT012, Khoa Kỹ thuật Máy tính, UIT).
- Slide bài giảng Chương 2 – Biểu diễn thông tin trong máy tính (số có dấu bù 2) và Chương 3 – Đại số Boole (bìa Karnaugh, chi phí C(B)).
- Tài liệu *Hướng dẫn sử dụng Logisim* do giảng viên cung cấp.